Table of Contents

- register
  - Parameters
- login
  - Parameters
- me
  - Parameters
- create
  - Parameters
- create
  - Parameters
- create
  - Parameters
- create
  - Parameters
- getAll
  - Parameters
- getAll
  - Parameters
- getAll
  - Parameters
- getAll
  - Parameters
- getAll
  - Parameters
- getOne
  - Parameters
- getByModel
  - Parameters
- getByModel
  - Parameters
- getByModel
  - Parameters
- update
  - Parameters
- update
  - Parameters
- update
  - Parameters
- update
  - Parameters
- remove
  - Parameters
- remove

    - Parameters
- remove
    - Parameters
- remove
    - Parameters
- remove
    - Parameters
- remove
    - Parameters
- addToModel
    - Parameters
- addToModel
    - Parameters
- removeFromModel
    - Parameters
- removeFromModel
    - Parameters
- reply
    - Parameters
- appendCleanFolderToArchive
    - Parameters
- record
    - Parameters
- getUserHistory
    - Parameters
- getModelStats
    - Parameters
- favorite
    - Parameters
- unfavorite
    - Parameters
- getUserFavoritesPublic
    - Parameters
- getMyFavorites
    - Parameters
- check
    - Parameters
- check
    - Parameters
- follow
    - Parameters
- unfollow
    - Parameters
- followers

    - Parameters
- following
    - Parameters
- formatUploadedFiles
    - Parameters
- uploadModel
    - Parameters
- getById
    - Parameters
- getById
    - Parameters
- getByUser
    - Parameters
- like
    - Parameters
- unlike
    - Parameters
- patchMainFile
    - Parameters
- patchMainImage
    - Parameters
- removeMainImage
    - Parameters
- uploadImage
    - Parameters
- getImages
    - Parameters
- updateOrder
    - Parameters
- removeImage
    - Parameters
- getForModel
    - Parameters
- getFavorites
    - Parameters
- getAllPublicUsers
    - Parameters
- verifyToken
    - Parameters
- optionalToken
    - Parameters
- isAdmin
    - Parameters
- isOwnerOrAdmin

    - Parameters
- getModelBaseDirectory
    - Parameters
- cleanFileName
    - Parameters
- createInitialStorage
    - Parameters
- createDynamicStorage
    - Parameters
- createFilter
    - Parameters
- uploadModelFile
- modelUploadFields
- uploadImageFile
- uploadPartsFile
- uploadMainImageFile
- uploadMainFileReplacement
- createUploadWrapper
    - Parameters
- registerUser
    - Parameters
- loginUser
    - Parameters
- getCurrentUser
    - Parameters
- createCategory
    - Parameters
- getCategories
- getCategoryById
    - Parameters
- updateCategory
    - Parameters
- deleteCategory
    - Parameters
- getCategoriesByModel
    - Parameters
- addCategoryToModel
    - Parameters
- removeCategoryFromModel
    - Parameters
- createComment
    - Parameters
- getModelComments
    - Parameters
- updateComment

    - Parameters
- deleteComment
    - Parameters
- replyToComment
    - Parameters
- recordDownload
    - Parameters
- getDownloadsHistory
    - Parameters
- getModelDownloadStats
    - Parameters
- getDownloadInfo
    - Parameters
- addFavorite
    - Parameters
- removeFavorite
    - Parameters
- getUserFavorites
    - Parameters
- getUserFavorites
    - Parameters
- checkFavorite
    - Parameters
- followUser
    - Parameters
- unfollowUser
    - Parameters
- getFollowers
    - Parameters
    - Properties
- getFollowing
    - Parameters
    - Properties
- checkFollow
    - Parameters
- createPart
    - Parameters
- getParts
    - Parameters
    - Properties
- getPartsByModelId
    - Parameters
- deletePart
    - Parameters

- createModel
  - Parameters
- getModelById
  - Parameters
- getModelIncludes
- getModels
  - Parameters
- getModelsByUser
  - Parameters
- deleteModel
  - Parameters
- updateModel
  - Parameters
- addLike
  - Parameters
- removeLike
  - Parameters
- updateMainImage
  - Parameters
- deleteMainImage
  - Parameters
- replaceMainFile
  - Parameters
- addImage
  - Parameters
- getModelImages
  - Parameters
- updateImageOrder
  - Parameters
- deleteImage
  - Parameters
- normalizeTag
  - Parameters
- getTagsForModel
  - Parameters
- addTagToModel
  - Parameters
- removeTagFromModel
  - Parameters
- removeTag
  - Parameters
- getAllTags
- getUserById
  - Parameters

- getUsers
  - Parameters
  - Properties
- getPublicUsers
  - Parameters
  - Properties
- updateUser
  - Parameters
- deleteUser
  - Parameters
- checkPermission
  - Parameters
- generateToken
  - Parameters
- getAbsolutePath
  - Parameters
- deletePhysicalFile
  - Parameters
- deletePhysicalFolder
  - Parameters
- getDisplayFileName
  - Parameters
- sendSuccess
  - Parameters
- sendError
  - Parameters
- validateUserFields
  - Parameters

register

Registra un nuevo usuario en la plataforma. Crea cuenta y devuelve datos básicos + token JWT.

Parameters

- `req`
- `res`

login

Autentica a un usuario e inicia sesión. Devuelve token JWT y datos básicos del usuario si las credenciales son válidas.

Parameters

- `req`
- `res`

me

Obtiene los datos completos del usuario autenticado actualmente. Útil para validar sesión activa y obtener información del perfil propio. Requiere token válido en Authorization header.

Parameters

- `req`
- `res`

create

Crea una nueva categoría en el sistema. Requiere nombre único.

Parameters

- `req`
- `res`

create

Crea un nuevo comentario en un modelo específico. Requiere autenticación.

Parameters

- `req`
- `res`

create

Crea una o varias piezas (parts) para un modelo a partir de archivos 3D subidos. Requiere autenticación y que el usuario sea propietario del modelo o admin. Genera automáticamente el nombre de la pieza desde el nombre del archivo.

Parameters

- `req`
- `res`

create

Crea el registro del modelo en la base de datos usando los datos enviados y las rutas preparadas. Requiere autenticación.

Parameters

- `req`
- `res`

getAll

Obtiene la lista completa de categorías ordenadas alfabéticamente. Endpoint público.

Parameters

- `req`
- `res`

getAll

Obtiene una lista paginada de todas las piezas registradas en el sistema. Útil principalmente para administración.

Parameters

- `req`
- `res`

getAll

Obtiene todos los modelos paginados de la plataforma. Endpoint público.

Parameters

- `req`
- `res`

getAll

Obtiene la lista completa de todos los tags existentes en la plataforma. Ordenados alfabéticamente. Endpoint público.

Parameters

- `req`
- `res`

getAll

Obtiene una lista paginada de todos los usuarios de la plataforma. Endpoint público (útil para búsqueda de creadores, rankings, etc.).

Parameters

- `req`
- `res`

getOne

Obtiene una categoría específica por su ID, incluyendo conteo de modelos asociados. Endpoint público.

Parameters

- `req`
- `res`

getByModel

Obtiene todas las categorías asignadas a un modelo específico. Endpoint público.

Parameters

- `req`
- `res`

getByModel

Obtiene los comentarios de un modelo de forma paginada. Ordenados por fecha descendente (más recientes primero). Endpoint público.

Parameters

- `req`
- `res`

getByModel

Obtiene todas las piezas asociadas a un modelo específico. Endpoint público.

Parameters

- `req`
- `res`

update

Actualiza el nombre de una categoría existente. El nombre debe seguir siendo único.

Parameters

- `req`
- `res`

update

Actualiza el contenido de un comentario existente. Requiere autenticación y que el usuario sea el autor del comentario o administrador.

Parameters

- `req`
- `res`

update

Actualiza los campos textuales y metadatos básicos de un modelo. Requiere ser propietario o administrador.

Parameters

- `req`
- `res`

update

Actualiza los datos públicos del perfil del usuario. Requiere autenticación y que el usuario sea el propietario o admin.

Parameters

- `req`
- `res`

remove

Elimina una categoría del sistema (las asociaciones con modelos se eliminan automáticamente por CASCADE).

Parameters

- `req`
- `res`

remove

Elimina un comentario específico. Requiere autenticación y que el usuario sea el autor del comentario o administrador.

Parameters

- `req`
- `res`

remove

Elimina una pieza específica (de la base de datos y del sistema de archivos). Requiere autenticación y que el usuario sea propietario del modelo o admin.

Parameters

- `req`
- `res`

remove

Elimina un modelo completo (BD + archivos físicos). Requiere ser propietario o administrador.

Parameters

- `req`
- `res`

remove

Elimina un tag permanentemente del sistema (incluyendo todas sus asociaciones). Requiere rol de administrador.

Parameters

- `req`
- `res`

remove

Elimina completamente un usuario y todos sus archivos asociados. Requiere autenticación y que el usuario sea el propietario o admin.

Parameters

- `req`

- `res`

addToModel

Asocia una categoría existente a un modelo. Requiere autenticación y que el usuario sea propietario del modelo o admin.

Parameters

- `req`
- `res`

addToModel

Añade un nuevo tag (o usa uno existente) a un modelo. Requiere autenticación y que el usuario sea propietario del modelo o admin.

Parameters

- `req`
- `res`

removeFromModel

Elimina la asociación entre una categoría y un modelo. Requiere autenticación y que el usuario sea propietario del modelo o admin.

Parameters

- `req`
- `res`

removeFromModel

Elimina la asociación de un tag específico con un modelo. Requiere autenticación y que el usuario sea propietario del modelo o admin.

Parameters

- `req`
- `res`

reply

Responde a un comentario existente. Requiere autenticación.

Parameters

- `req`
- `res`

appendCleanFolderToArchive

Añade recursivamente los archivos de una carpeta a un ZIP, limpiando nombres de archivo. Ignora subcarpetas (solo archivos directos).

Parameters

- `archive` **archiver.Archiver** Instancia del ZIP en curso
- `folderPath` **string** Ruta absoluta de la carpeta a comprimir
- `zipSubFolder` **string** Subcarpeta dentro del ZIP (ej: "parts", "gallery") (optional, default `""`)

record

Registra la descarga y devuelve el archivo solicitado (individual o ZIP comprimido). Soporta tipos: main (por defecto), all, parts, gallery. Limpia nombres de archivo y registra IP, user-agent y usuario (si autenticado).

Parameters

- `req`
- `res`

getUserHistory

Obtiene el historial paginado de descargas del usuario autenticado. Requiere autenticación.

Parameters

- `req`
- `res`

getModelStats

Obtiene estadísticas detalladas de descargas de un modelo (total, únicos, anónimos, etc.). Requiere ser propietario del modelo o administrador.

Parameters

- `req`
- `res`

favorite

Añade un modelo a los favoritos del usuario autenticado. Idempotente: si ya estaba marcado como favorito, devuelve mensaje informativo. Requiere autenticación.

Parameters

- `req`
- `res`

unfavorite

Elimina un modelo de los favoritos del usuario autenticado. Idempotente: si no estaba en favoritos, devuelve mensaje informativo. Requiere autenticación.

Parameters

- `req`
- `res`

getUserFavoritesPublic

Obtiene la lista paginada de favoritos de un usuario específico. Endpoint público (cualquiera puede ver los favoritos de otros usuarios).

Parameters

- `req`
- `res`

getMyFavorites

Obtiene la lista paginada de favoritos del usuario autenticado. Requiere autenticación.

Parameters

- `req`
- `res`

check

Comprueba si un modelo está marcado como favorito por el usuario autenticado. Requiere autenticación.

Parameters

- `req`
- `res`

check

Comprueba si el usuario autenticado sigue al usuario indicado. Requiere autenticación.

Parameters

- `req`
- `res`

follow

Permite al usuario autenticado seguir a otro usuario. Idempotente: si ya se seguía, devuelve mensaje informativo. Requiere autenticación.

Parameters

- `req`
- `res`

unfollow

Permite al usuario autenticado dejar de seguir a otro usuario. Idempotente: si no se seguía, devuelve mensaje informativo. Requiere autenticación.

Parameters

- `req`

- `res`

followers

Obtiene la lista paginada de seguidores de un usuario específico. Endpoint público.

Parameters

- `req`
- `res`

following

Obtiene la lista paginada de usuarios a los que sigue un usuario específico. Endpoint público.

Parameters

- `req`
- `res`

formatUploadedFiles

Formatea las rutas de los archivos subidos por Multer para que coincidan con la estructura esperada por la base de datos. Valida que exista el archivo principal 3D.

Parameters

- `files` **Object** Objeto de archivos subidos por Multer (req.files).
- `userId` **string** ID del usuario que sube.
- `uploadId` **(number | string)** Identificador temporal de la subida.

- Throws **Error** Si falta el archivo principal (main_file).

Returns **Object** Objeto con rutas formateadas: main_file, cover_image, parts y gallery.

uploadModel

Prepara y devuelve las rutas de los archivos subidos en la creación inicial del modelo. No guarda en BD aún (solo organiza archivos).

Parameters

- `req`
- `res`

getById

Obtiene los datos completos de un modelo por su ID (incluye relaciones). Endpoint público.

Parameters

- `req`
- `res`

getById

Obtiene el perfil completo público de un usuario por ID. Cualquiera puede verlo (sin email ni datos privados).

Parameters

- `req`
- `res`

getByUser

Obtiene los modelos paginados creados por un usuario específico. Endpoint público.

Parameters

- `req`
- `res`

like

Registra un like del usuario autenticado al modelo. Idempotente (si ya existe, no falla).

Parameters

- `req`
- `res`

unlike

Retira el like del usuario autenticado al modelo. Idempotente (si no existía, no falla).

Parameters

- `req`
- `res`

patchMainFile

Reemplaza el archivo 3D principal del modelo. Borra el archivo anterior del disco. Requiere ser propietario o admin.

Parameters

- `req`
- `res`

patchMainImage

Reemplaza la imagen de portada (main_image) del modelo. Borra la imagen anterior del disco. Requiere ser propietario o admin.

Parameters

- `req`
- `res`

removeMainImage

Elimina la imagen de portada del modelo (BD + disco). Requiere ser propietario o admin.

Parameters

- `req`
- `res`

uploadImage

Sube una o varias imágenes a la galería secundaria de un modelo. Requiere autenticación y que el usuario sea propietario o admin. Crea entradas en la tabla model_images con orden secuencial.

Parameters

- `req`
- `res`

getImages

Obtiene todas las imágenes de la galería de un modelo, ordenadas por display_order. Endpoint público.

Parameters

- `req`
- `res`

updateOrder

Actualiza el orden de visualización (display_order) de una imagen específica. Requiere autenticación y permiso sobre el modelo.

Parameters

- `req`
- `res`

removeImage

Elimina una imagen de la galería (de la base de datos y del sistema de archivos). Requiere autenticación y permiso sobre el modelo.

Parameters

- `req`
- `res`

getForModel

Obtiene todos los tags asociados a un modelo específico. Endpoint público.

Parameters

- `req`
- `res`

getFavorites

Obtiene la lista de modelos marcados como favoritos por un usuario. No requiere autenticación (público).

Parameters

- `req`
- `res`

getAllPublicUsers

Obtiene lista pública de usuarios (tarjetas para página "Desarrolladores"). Solo campos seguros y visibles.

Parameters

- `req`
- `res`

verifyToken

Middleware obligatorio: verifica que exista un token Bearer válido. Si es válido, adjunta el payload decodificado en `req.user`. Si falla, responde con 401 o 403 y detiene la cadena.

Parameters

- `req`
- `res`
- `next`

optionalToken

Middleware opcional: intenta verificar el token Bearer si existe. Adjunta `req.user` (objeto decodificado o `null` si no hay token o es inválido). Nunca falla la petición, siempre llama a `next()`.

Parameters

- `req`
- `res`
- `next`

isAdmin

Middleware de autorización: requiere que el usuario esté autenticado y tenga rol "admin". Responde 403 si no es administrador.

Parameters

- `req`
- `res`
- `next`

isOwnerOrAdmin

Middleware de autorización: permite el acceso solo si el usuario autenticado es el propietario del recurso (por ID en params) o es administrador. Usa `req.params.id` como ID del recurso a proteger.

Parameters

- `req`

- `res`
- `next`

getModelBaseDirectory

Obtiene la carpeta base física de un modelo (donde están main_file, gallery, parts, etc.) a partir de la ruta almacenada en la BD.

Parameters

- `modelId` **string**
- Throws **Error** Si el modelo no existe

Returns **Promise<string>** Ruta absoluta al directorio base del modelo

cleanFileName

Limpia y normaliza el nombre del archivo (reemplaza espacios por _)

Parameters

- `name` **string** Nombre original del archivo
- `prefix` **string** Prefijo opcional (ej: "cover_", "main_") (optional, default `""`)

Returns **string** Nombre limpio

createInitialStorage

Storage de Multer para la creación inicial de un modelo (subida múltiple de archivos). Crea carpetas dinámicas: /uploads/models/:userId/:timestamp/[parts|gallery]

Type: multer.StorageEngine

Parameters

- `folder`

createDynamicStorage

Storage dinámico para subir/reemplazar archivos en modelos ya existentes. Usa la ruta almacenada en BD para determinar el destino.

Parameters

- `subFolder` **string** Subcarpeta adicional (gallery, parts, etc.) (optional, default `""`)
- `checkExists` **boolean** Verificar si el archivo ya existe (optional, default `false`)
- `prefix` **string** Prefijo para el nombre del archivo (optional, default `""`)
- `useTimestamp` **boolean** Añadir timestamp para evitar sobreescritura accidental (optional, default `false`)

Returns **multer.StorageEngine**

createFilter

Crea filtro de Multer para validar extensiones permitidas

Parameters

- `allowedTypes` **Array**<**string**> Extensiones permitidas (con punto)
- `errorMsg` **string** Mensaje de error personalizado

Returns **multer.FileFilter**

uploadModelFile

Subida inicial completa de modelo (main + cover + parts + gallery)

modelUploadFields

Configuración de campos múltiples para subida inicial

uploadImageFile

Añadir múltiples imágenes a la galería (máx 10)

uploadPartsFile

Añadir múltiples partes 3D (máx 10)

uploadMainImageFile

Reemplazar imagen de portada (única)

uploadMainFileReplacement

Reemplazar archivo principal 3D (único)

createUploadWrapper

Wrapper que captura errores comunes de Multer y responde con JSON 400

Parameters

- `uploadFn` **Function** Función multer (single/array/fields)
- `limitErrorMsg` **string?** Mensaje personalizado para LIMIT_UNEXPECTED_FILE

registerUser

Registra un nuevo usuario en la plataforma.

Parameters

- `userData` **Object** Datos del usuario (name, lastname, username, email, password).
  - `userData.name`
  - `userData.lastname`
  - `userData.username`
  - `userData.email`

- `userData.password`

Returns **Promise**<**Object**> Objeto con los datos públicos del usuario y su token JWT.

loginUser

Autentica a un usuario verificando sus credenciales.

Parameters

- `credentials` **Object** Email y contraseña del usuario.
  - `credentials.email`
  - `credentials.password`

Returns **Promise**<**Object**> Objeto con los datos públicos del usuario y su token JWT.

getCurrentUser

Obtiene los datos básicos del usuario actualmente autenticado (usado para validar sesiones).

Parameters

- `userId` **string** ID del usuario extraído del token.

Returns **Promise**<**Object**> Datos del usuario (excluyendo datos sensibles).

createCategory

Crea una nueva categoría en el sistema. El nombre debe ser único (restringido por índice único en la base de datos).

Parameters

- `name` **string** Nombre de la categoría (obligatorio)
- Throws **Error** Si el nombre está vacío o no se proporciona
- Throws **Error** Si ya existe una categoría con ese nombre (error P2002 de Prisma)

Returns **Promise**<**Object**> Categoría recién creada

getCategories

Obtiene la lista completa de categorías ordenadas alfabéticamente por nombre.

Returns **Promise**<**Array**<**{id: string, name: string, created_at: Date, updated_at: Date}**>> Lista de todas las categorías

getCategoryById

Obtiene una categoría por su ID, incluyendo el conteo de modelos que la utilizan.

Parameters

- `id` **string** ID de la categoría
- Throws **Error** "Categoría no encontrada" si no existe

Returns **Promise**<**Object**> Categoría con propiedad _count.model_category

updateCategory

Actualiza el nombre de una categoría existente.

Parameters

- `id` **string** ID de la categoría a actualizar
- `name` **string** Nuevo nombre de la categoría

- Throws **Error** "Categoría no encontrada" si el ID no existe (P2025)
- Throws **Error** Si ya existe otra categoría con el nuevo nombre (P2002)

Returns **Promise**<**Object**> Categoría actualizada

deleteCategory

Elimina una categoría del sistema. Gracias a la restricción ON DELETE CASCADE en model_category, las relaciones con modelos se eliminan automáticamente.

Parameters

- `id` **string** ID de la categoría a eliminar

- Throws **Error** "Categoría no encontrada" si el ID no existe (P2025)

Returns **Promise**<**{message: string}**> Mensaje de confirmación

getCategoriesByModel

Obtiene todas las categorías asignadas a un modelo específico.

Parameters

- `modelId` **string** ID del modelo

Returns **Promise**<**Array**<**{id: string, name: string}**>> Lista de categorías del modelo

addCategoryToModel

Asocia una categoría existente a un modelo. Solo el propietario del modelo o un administrador puede ejecutar esta acción.

Parameters

- `modelId` **string** ID del modelo
- `categoryId` **string** ID de la categoría a asignar
- `user` **Object** Usuario autenticado que realiza la acción

- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si el usuario no tiene permiso sobre el modelo

Returns **Promise**<**{message: string}**> Resultado de la operación

removeCategoryFromModel

Elimina la asociación entre una categoría y un modelo. Solo el propietario del modelo o un administrador puede ejecutar esta acción.

Parameters

- `modelId` **string** ID del modelo
- `categoryId` **string** ID de la categoría a desasociar
- `user` **Object** Usuario autenticado que realiza la acción

- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si el usuario no tiene permiso sobre el modelo

Returns **Promise**<**{message: string}**> Resultado de la operación

createComment

Crea un nuevo comentario en un modelo.

Parameters

- `modelId` **string** ID del modelo comentado.
- `userId` **string** ID del usuario que comenta.
- `content` **string** Contenido del comentario.

Returns **Promise**<**Object**> El comentario recién creado.

getModelComments

Obtiene los comentarios de un modelo de forma paginada junto con los datos del autor.

Parameters

- `modelId` **string** ID del modelo.

- `pagination` **Object** Objeto con page y limit.

  - `pagination.page` (optional, default `1`)
  - `pagination.limit` (optional, default `20`)

Returns **Promise**<**Object**> Resultado paginado con los comentarios.

updateComment

Actualiza el contenido de un comentario.

Parameters

- `commentId` **string** ID del comentario.
- `user` **Object** Usuario que realiza la petición (para verificar permisos).
- `content` **string** Nuevo texto del comentario.

Returns **Promise**<**Object**> El comentario actualizado.

deleteComment

Elimina un comentario.

Parameters

- `commentId` **string** ID del comentario.
- `user` **Object** Usuario que realiza la petición.

Returns **Promise<Object>** Mensaje de éxito.

replyToComment

Crea una respuesta a un comentario existente. Hereda automáticamente el model_id del comentario padre.

- @param {string} commentId - ID del comentario al que se va a responder (comentario padre).

Parameters

- `commentId`
- `userId` **string** ID del usuario que escribe la respuesta.
- `content` **string** Texto del contenido de la respuesta.

Returns **Promise<Object>** El nuevo comentario (respuesta) creado.

recordDownload

Registra una descarga del modelo en la base de datos y actualiza el contador de descargas totales del modelo de forma atómica (transacción). Devuelve también la URL del archivo para que el cliente pueda proceder con la descarga.

Parameters

- `modelId` **string** ID del modelo que se está descargando
- `user` **(Object | null)** Objeto del usuario autenticado (null si es descarga anónima)
- `ip` **string** Dirección IP del cliente (para registro y prevención de abuso)
- `userAgent` **string** Cadena User-Agent del navegador o cliente

- Throws **Error** "El modelo solicitado no existe" (código P2025)
- Throws **Error** Error genérico de base de datos si ocurre otro problema

Returns **Promise<{message: string, downloads_count: number, file_url: string}>** Información de confirmación y datos necesarios para la descarga

getDownloadsHistory

Obtiene el historial paginado de descargas realizadas por un usuario específico. Solo muestra descargas registradas (no anónimas).

Parameters

- `userId` **string** ID del usuario cuyos descargas se consultan

- `options` **Object?** Opciones de paginación (optional, default `{}`)

  - `options.page` **number** Página solicitada (base 1) (optional, default `1`)
  - `options.limit` **number** Cantidad de registros por página (máx. 50) (optional, default `20`)

getModelDownloadStats

Obtiene estadísticas detalladas de descargas de un modelo específico. Solo accesible por el creador del modelo o administradores. Incluye total, descargas registradas, usuarios únicos y descargas anónimas.

Parameters

- `modelId` **string** ID del modelo cuyas estadísticas se consultan
- `user` **Object** Usuario autenticado que realiza la consulta
- Throws **Error** "El modelo solicitado no existe"
- Throws **Error** Si el usuario no tiene permiso (propietario o admin)

Returns **Promise<{total_downloads: number, unique_users: number, registered_downloads: number, anonymous_downloads: number}>** Estadísticas de descargas

getDownloadInfo

Obtiene información necesaria para servir el archivo de descarga: ruta absoluta en disco, nombre limpio sugerido y carpeta del modelo. Verifica que el archivo físico exista antes de retornar.

Parameters

- `modelId` **string** ID del modelo cuyo archivo se va a descargar
- Throws **Error** "El modelo solicitado no existe"
- Throws **Error** "El archivo físico no existe en el servidor" si el archivo no se encuentra

Returns **Promise<{absolutePath: string, cleanName: string, modelFolder: string}>** Información para servir el archivo

addFavorite

Añade un modelo a la lista de favoritos de un usuario.

Parameters

- `modelId` **string** ID del modelo.
- `userId` **string** ID del usuario.

removeFavorite

Elimina un modelo de la lista de favoritos de un usuario.

Parameters

- `modelId` **string** ID del modelo.
- `userId` **string** ID del usuario.

getUserFavorites

Obtiene la lista paginada de modelos favoritos de un usuario.

Parameters

- `userId` **string** ID del usuario del que se quieren ver los favoritos.
- `pagination` **Object** Opciones de paginación { page, limit }.

- `pagination.page` (optional, default `1`)
- `pagination.limit` (optional, default `20`)

getUserFavorites

Obtiene la lista completa de modelos que el usuario ha marcado como favoritos. Devuelve solo la información relevante de cada modelo.

Parameters

- `userId` **string** ID del usuario cuyos favoritos se desean obtener

Returns **Promise<Array<Object>>** Array de modelos favoritos

checkFavorite

Verifica si un usuario tiene un modelo en sus favoritos.

Parameters

- `modelId` **string** ID del modelo.
- `userId` **string** ID del usuario.

followUser

Permite que un usuario siga a otro usuario. Actualiza los contadores de seguidores y seguidos en una transacción. Es idempotente: si ya se seguía, retorna mensaje informativo sin error.

Parameters

- `userIdToFollow` **string** ID del usuario que se desea seguir
- `followerId` **string** ID del usuario que realiza la acción (el seguidor)

- Throws **Error** Si se intenta seguir a uno mismo
- Throws **Error** Error genérico de base de datos (excepto duplicado P2002)

Returns **Promise<{message: string}>** Mensaje de resultado

unfollowUser

Permite que un usuario deje de seguir a otro. Actualiza los contadores de seguidores y seguidos en una transacción. Es idempotente: si no se seguía, retorna mensaje informativo sin error.

Parameters

- `userIdToUnfollow` **string** ID del usuario que se desea dejar de seguir
- `followerId` **string** ID del usuario que realiza la acción

- Throws **Error** Error genérico de base de datos (excepto no encontrado P2025)

Returns **Promise<{message: string}>** Mensaje de resultado

getFollowers

Obtiene la lista paginada de usuarios que siguen a un usuario específico. Devuelve información básica del seguidor + fecha en que comenzó a seguir.

Parameters

- `userId` **string** ID del usuario cuyos seguidores se quieren obtener
- `options` **Object?** Opciones de paginación (optional, default `{}`)
  - `options.page` **number** Página solicitada (base 1) (optional, default `1`)
  - `options.limit` **number** Cantidad de registros por página (máx 50) (optional, default `20`)

Properties

- `page` **number** Página actual
- `limit` **number** Registros por página
- `total` **number** Total de seguidores
- `totalPages` **number** Total de páginas

Returns **Promise<Object>** Resultado paginado

getFollowing

Obtiene la lista paginada de usuarios que un usuario específico está siguiendo. Devuelve información básica del usuario seguido + fecha en que se comenzó a seguir.

Parameters

- `userId` **string** ID del usuario cuyos seguidos se quieren obtener
- `options` **Object?** Opciones de paginación (optional, default `{}`)
  - `options.page` **number** Página solicitada (optional, default `1`)
  - `options.limit` **number** Cantidad de registros por página (máx 50) (optional, default `20`)

Properties

- `page` **number** Página actual
- `limit` **number** Registros por página
- `total` **number** Total de usuarios seguidos
- `totalPages` **number** Total de páginas

Returns **Promise<Object>** Resultado paginado

checkFollow

Verifica si un usuario está siguiendo a otro usuario específico.

Parameters

- `userIdToCheck` **string** ID del usuario que podría estar siendo seguido
- `followerId` **string** ID del posible seguidor

Returns **Promise<{isFollowing: boolean}>** Resultado de la verificación

createPart

Añade una nueva pieza (parte/componente) a un modelo existente. Solo el propietario del modelo o un administrador puede realizar esta acción.

Parameters

- `user` **Object** Usuario autenticado que realiza la acción
- `modelId` **string** ID del modelo al que se añadirá la pieza
- `data` **Object** Datos de la pieza a crear
  - `data.color` **string?** Color de la pieza (opcional)
  - `data.part_name` **string** Nombre de la pieza (obligatorio)
  - `data.file_url` **string** URL/ruta del archivo de la pieza (obligatorio)
  - `data.file_size` **number?** Tamaño del archivo en bytes (opcional)
- Throws **Error** "Modelo no encontrado" si el modelo no existe
- Throws **Error** Si el usuario no tiene permiso sobre el modelo
- Throws **Error** Si faltan campos obligatorios (part_name o file_url)

Returns **Promise**<**Object**> La pieza recién creada

getParts

Obtiene una lista paginada de TODAS las piezas registradas en el sistema. Útil principalmente para propósitos administrativos o auditoría.

Parameters

- `options` **Object?** Opciones de paginación
  - `options.page` **number** Número de página (base 1) (optional, default `1`)
  - `options.limit` **number** Cantidad de registros por página (máximo 50) (optional, default `20`)

Properties

- `page` **number** Página actual
- `limit` **number** Registros por página
- `total` **number** Total de piezas en el sistema
- `totalPages` **number** Total de páginas disponibles
- `data` **Array**<**Object**> Lista de piezas

Returns **Promise**<**Object**> Objeto con paginación y lista de piezas

getPartsByModelId

Obtiene todas las piezas asociadas a un modelo específico, ordenadas por fecha de creación ascendente.

Parameters

- `modelId` **string** ID del modelo cuyas piezas se desean obtener

Returns **Promise**<**Array**<**Object**>> Lista de piezas del modelo

deletePart

Elimina una pieza específica tanto de la base de datos como del sistema de archivos. Solo el propietario del modelo o un administrador puede eliminarla.

Parameters

- `partId` **string** ID de la pieza a eliminar
- `user` **Object** Usuario autenticado que realiza la acción

- Throws **Error** "Parte no encontrada" si la pieza no existe
- Throws **Error** Si el usuario no tiene permiso sobre el modelo propietario

Returns **Promise**<**{message: string}**> Mensaje de confirmación

createModel

Crea un nuevo modelo en la base de datos junto con sus relaciones: partes (model_parts), imágenes de galería (model_images), etiquetas (model_tag) y categorías (model_category).

Parameters

- `userId` **string** ID del usuario creador del modelo

- `data` **Object** Datos del modelo y sus relaciones

  - `data.title` **string** Título del modelo (obligatorio)
  - `data.main_color` **string?** Color principal (hex o nombre)
  - `data.description` **string?** Descripción detallada
  - `data.file_url` **string** URL del archivo principal 3D
  - `data.main_image_url` **string?** URL de la imagen de portada
  - `data.video_url` **string?** URL de video demostración (opcional)
  - `data.license` **string?** Licencia (por defecto: "All Rights Reserved")
  - `data.parts` **Array**<**Object**>**?** Lista de partes/componentes
  - `data.images` **Array**<**string**>**?** Lista de URLs de imágenes (alternativa a gallery)
  - `data.gallery` **Array**<**string**>**?** Lista de URLs de imágenes (nombre usado por frontend)
  - `data.tags` **Array**<**string**>**?** IDs de etiquetas existentes
  - `data.categories` **Array**<**string**>**?** IDs de categorías existentes

- Throws **Error** Si faltan datos obligatorios o hay error en la transacción

Returns **Promise**<**Object**> Modelo creado con sus datos básicos (sin relaciones expandidas)

getModelById

Obtiene un modelo por su ID, incrementa el contador de vistas en 1 y devuelve el modelo completo con todas sus relaciones principales.

Parameters

- `modelId` **string** ID del modelo a consultar

- Throws **Error** "Modelo no encontrado" si no existe

Returns **Promise**<**Object**> Modelo con relaciones: creador, partes, imágenes ordenadas, tags, categorías y conteo de likes

getModelIncludes

Devuelve el objeto de inclusión (include) estándar para consultas de listado de modelos. Centraliza las relaciones más usadas para evitar repetir código.

Returns **Object** Configuración de include para prisma.models.findMany / findUnique

getModels

Obtiene una lista paginada de todos los modelos públicos, ordenados por fecha de creación descendente (más recientes primero).

Parameters

- `options` **Object?** Opciones de paginación
  - `options.page` **number** Página solicitada (base 1) (optional, default `1`)
  - `options.limit` **number** Elementos por página (máx. 50) (optional, default `20`)

Returns **Promise<Object>** Objeto paginado con metadata y lista de modelos

getModelsByUser

Obtiene una lista paginada de los modelos creados por un usuario específico.

Parameters

- `userId` **string** ID del usuario propietario
- `options` **Object?** Opciones de paginación
  - `options.page` **number** Página solicitada (optional, default `1`)
  - `options.limit` **number** Elementos por página (máx. 50) (optional, default `20`)

Returns **Promise<Object>** Objeto paginado con metadata y lista de modelos del usuario

deleteModel

Elimina un modelo de la base de datos y su carpeta física completa (incluyendo archivo principal y todas las imágenes asociadas). Requiere permisos de propietario o administrador.

Parameters

- `modelId` **string** ID del modelo a eliminar
- `user` **Object** Usuario autenticado que ejecuta la acción
- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si no tiene permisos suficientes

Returns **Promise<{message: string}>** Mensaje de confirmación

updateModel

Actualiza los campos textuales y de metadatos básicos del modelo. No maneja archivos ni relaciones complejas (partes, imágenes, tags, etc).

Parameters

- `modelId` **string** ID del modelo a actualizar
- `user` **Object** Usuario autenticado (debe ser propietario)

- `data` **Object** Campos a actualizar
  - `data.title` **string?** Nuevo título
  - `data.description` **string?** Nueva descripción
  - `data.main_color` **string?** Nuevo color principal
  - `data.license` **string?** Nueva licencia
  - `data.video_url` **string?** Nueva URL de video
- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si no tiene permisos

Returns **Promise<Object>** Modelo actualizado

addLike

Registra un like del usuario al modelo (si no existe ya). Idempotente: si ya existe el like, no hace nada.

Parameters

- `modelId` **string** ID del modelo
- `userId` **string** ID del usuario que da like

Returns **Promise<{likes: number}>** Cantidad actual de likes

removeLike

Elimina el like de un usuario a un modelo (si existe). Idempotente: si no existía, no hace nada.

Parameters

- `modelId` **string** ID del modelo
- `userId` **string** ID del usuario que retira el like

Returns **Promise<{likes: number}>** Cantidad actual de likes

updateMainImage

Actualiza la imagen principal (portada) del modelo. Borra físicamente la imagen anterior si existía y era diferente.

Parameters

- `modelId` **string** ID del modelo
- `user` **Object** Usuario autenticado (debe ser propietario)
- `imageUrl` **string** Nueva URL de la imagen principal
- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si no tiene permisos

Returns **Promise<Object>** Modelo actualizado

deleteMainImage

Elimina la imagen principal del modelo tanto de la BD como del sistema de archivos.

Parameters

- `modelId` **string** ID del modelo
- `user` **Object** Usuario autenticado (debe ser propietario)

- Throws **Error** "Modelo no encontrado"
- Throws **Error** "El modelo ya no tiene imagen principal"
- Throws **Error** Si no tiene permisos

Returns **Promise**<**{message: string}**> Mensaje de confirmación

replaceMainFile

Reemplaza el archivo principal 3D del modelo. Borra físicamente el archivo anterior si existía y era diferente.

Parameters

- `modelId` **string** ID del modelo
- `user` **Object** Usuario autenticado (debe ser propietario)
- `newFileUrl` **string** Nueva URL del archivo 3D

- Throws **Error** "Modelo no encontrado"
- Throws **Error** Si no tiene permisos

Returns **Promise**<**Object**> Modelo actualizado

addImage

Añade una nueva imagen a la galería de un modelo.

Parameters

- `user` **Object** Usuario autenticado (para verificar permisos).
- `modelId` **string** ID del modelo padre.
- `imageUrl` **string** Ruta de la imagen subida.
- `displayOrder` **number** Orden de visualización en la galería. (optional, default `0`)

Returns **Promise**<**Object**> El registro de la imagen creada.

getModelImages

Obtiene todas las imágenes de la galería de un modelo.

Parameters

- `modelId` **string** ID del modelo.

Returns **Promise**<**Array**> Lista de imágenes ordenadas por display_order y fecha.

updateImageOrder

Actualiza el orden de visualización de una imagen en la galería.

Parameters

- `imageId` **string** ID de la imagen a actualizar.
- `user` **Object** Usuario autenticado.
- `newDisplayOrder` **number** Nuevo número de orden.

Returns **Promise<Object>** La imagen actualizada.

deleteImage

Elimina una imagen de la galería, tanto de la DB como del disco duro.

Parameters

- `imageId` **string** ID de la imagen a eliminar.
- `user` **Object** Usuario autenticado.

Returns **Promise<Object>** Mensaje de confirmación.

normalizeTag

Normaliza el nombre de un tag: elimina espacios al inicio y final y convierte todo a minúsculas. Se usa para mantener consistencia en los nombres de tags.

Parameters

- `name` **string** Nombre del tag a normalizar

Returns **string** El nombre normalizado (minúsculas, sin espacios sobrantes)

getTagsForModel

Obtiene todos los tags asociados a un modelo específico.

Parameters

- `modelId` **string** ID del modelo cuyos tags se desean recuperar

Returns **Promise<Array<{id: string, name: string}>>** Lista de tags asociados al modelo, ordenados alfabéticamente

addTagToModel

Añade un tag a un modelo. Si el tag no existe en el sistema, lo crea automáticamente. Solo el propietario del modelo o un administrador puede ejecutar esta acción.

Parameters

- `modelId` **string** ID del modelo al que se añadirá el tag
- `user` **Object** Objeto del usuario autenticado que realiza la acción
- `tagName` **string** Nombre del tag a añadir (se normalizará internamente)

- Throws **Error** "Modelo no encontrado" si el modelo no existe
- Throws **Error** Si el usuario no tiene permiso sobre el modelo
- Throws **Error** Error genérico de base de datos si ocurre otro problema

removeTagFromModel

Elimina la relación entre un tag y un modelo específico. Solo el propietario del modelo o un administrador puede realizar esta acción.

Parameters

- `modelId` **string** ID del modelo del que se quitará el tag
- `tagId` **string** ID del tag a remover del modelo
- `user` **Object** Objeto del usuario autenticado que realiza la acción

- Throws **Error** "Modelo no encontrado" si el modelo no existe
- Throws **Error** Si el usuario no tiene permiso sobre el modelo
- Throws **Error** Error genérico de base de datos si ocurre otro problema

Returns **Promise<{message: string}>** Mensaje de resultado de la operación

removeTag

Elimina un tag completamente del sistema (incluyendo todas sus relaciones con modelos). Acción restringida exclusivamente a usuarios con rol de administrador.

Parameters

- `tagId` **string** ID del tag a eliminar
- `user` **Object** Objeto del usuario autenticado (debe ser admin)

- Throws **Error** "Acción permitida solo para administradores" si el usuario no es admin
- Throws **Error** "Tag no encontrado" si el tag no existe
- Throws **Error** Error genérico de base de datos si ocurre otro problema

Returns **Promise<{message: string}>** Mensaje de confirmación

getAllTags

Obtiene la lista completa de todos los tags existentes en el sistema. Útil para autocompletado, listas desplegables o mostrar tags populares.

Returns **Promise<Array<{id: string, name: string}>>** Lista de todos los tags, ordenados alfabéticamente

getUserById

Obtiene el perfil completo de un usuario, incluyendo estadísticas globales de su impacto. Esta función trae absolutamente todo lo necesario para mostrar un perfil completo en el frontend: datos personales, redes sociales, estadísticas agregadas y contenido reciente (modelos propios y favoritos).

Parameters

- `userId` **string** ID del usuario cuyo perfil se desea obtener

- Throws **Error** Si el usuario no existe

Returns **Promise<Object>** Objeto con tres secciones principales: profile, stats y content

getUsers

Obtiene una lista paginada de usuarios con información básica y conteo de modelos. Ordenados por fecha de creación descendente (los más recientes primero).

Parameters

- `options` **Object?** Opciones de paginación
  - `options.page` **number** Número de página (comienza en 1) (optional, default `1`)

- `options.limit` **number** Cantidad de usuarios por página (máximo 50) (optional, default `20`)

Properties

- `page` **number** Página actual
- `limit` **number** Registros por página
- `total` **number** Total de usuarios en la base de datos
- `totalPages` **number** Total de páginas disponibles
- `data` **Array**<**Object**> Lista de usuarios

Returns **Promise**<**Object**> Objeto con paginación y lista de usuarios

getPublicUsers

Obtiene una lista paginada de usuarios con información básica y conteo de modelos. Ordenados por fecha de creación descendente (los más recientes primero).

Parameters

- `options` **Object?** Opciones de paginación
  - `options.page` **number** Número de página (comienza en 1) (optional, default `1`)
  - `options.limit` **number** Cantidad de usuarios por página (máximo 50) (optional, default `20`)

Properties

- `page` **number** Página actual
- `limit` **number** Registros por página
- `total` **number** Total de usuarios en la base de datos
- `totalPages` **number** Total de páginas disponibles
- `data` **Array**<**Object**> Lista de usuarios

Returns **Promise**<**Object**> Objeto con paginación y lista de usuarios

updateUser

Actualiza los datos de un usuario. Solo permite modificar campos permitidos. Requiere que el usuario autenticado tenga permiso para modificar el perfil.

Parameters

- `userId` **string** ID del usuario a actualizar
- `currentUser` **Object** Usuario autenticado que realiza la acción
- `data` **Object** Objeto con los campos a actualizar
  - `data.name` **string?** Nombre
  - `data.lastname` **string?** Apellido
  - `data.username` **string?** Nombre de usuario (único)
  - `data.avatar` **string?** URL del avatar
  - `data.bio` **string?** Biografía
  - `data.youtube` **string?** URL de YouTube
  - `data.twitter` **string?** URL de Twitter/X
  - `data.linkedin` **string?** URL de LinkedIn

    - `data.github` **string?** URL de GitHub
    - `data.location` **string?** Ubicación

- Throws **Error** Si no hay campos para actualizar
- Throws **Error** Si el nombre de usuario ya está en uso (código P2002)
- Throws **Error** Si el usuario no existe (código P2025)

Returns **Promise**<**Object**> Usuario actualizado con campos seleccionados

deleteUser

Elimina completamente a un usuario y todos sus archivos asociados en el sistema de archivos (carpetas de modelos e imágenes en /uploads). Requiere permisos de administración o ser el propio usuario.

Parameters

- `userId` **string** ID del usuario a eliminar
- `currentUser` **Object** Usuario autenticado que realiza la acción

- Throws **Error** Si el usuario no existe

Returns **Promise**<**Object**> Mensaje de confirmación

checkPermission

Verifica si el usuario actual tiene permiso para actuar sobre un recurso. Permite la acción si el usuario es propietario o tiene rol de administrador. Puedes extender la lógica agregando más condiciones en el futuro.

Parameters

- `resourceOwnerId` **string** ID del propietario del recurso

- `currentUser` **Object** Objeto del usuario autenticado

    - `currentUser.id` **string** ID del usuario
    - `currentUser.role` **string** Rol del usuario ("user", "admin", etc.)

- Throws **Error** Con status 403 si no tiene permiso

generateToken

Genera un token JWT firmado con el payload proporcionado. Incluye validación básica de variables de entorno y manejo seguro de expiración.

Parameters

- `payload` **Object** Información del usuario a codificar (id, username, role, etc.)

- `options` **Object?** Opciones adicionales (no recomendado sobreescribir expiresIn normalmente) (optional, default `{}`)

    - `options.expiresIn` **string?** Tiempo de expiración personalizado (ej: "1h", "30d")

- Throws **Error** Si falta JWT_SECRET o hay error durante la generación

Returns **string** Token JWT válido

getAbsolutePath

Convierte una URL relativa almacenada en la BD (ej: /uploads/models/...) en una ruta absoluta del sistema de archivos.

Parameters

- `fileUrl` **(string | null)** URL relativa del archivo (puede empezar con / o no)

Returns **(string | null)** Ruta absoluta completa o null si no se proporciona URL

deletePhysicalFile

Elimina un archivo físico del disco de forma segura. No lanza error si el archivo no existe o falla la eliminación (solo registra en consola).

Parameters

- `fileUrl` **string** URL relativa del archivo a eliminar

Returns **boolean** true si se eliminó correctamente, false si no existía o falló

deletePhysicalFolder

Elimina una carpeta completa y todo su contenido recursivamente. Útil al borrar un modelo entero (incluye main file, parts, gallery, etc.). No lanza error si la carpeta no existe o falla (solo registra en consola).

Parameters

- `fileUrl` **string** URL relativa de cualquier archivo dentro de la carpeta a eliminar

Returns **boolean** true si se eliminó la carpeta, false si no existía o falló

getDisplayFileName

Limpia el nombre del archivo para mostrarlo al usuario o en descargas. Elimina prefijos internos como "main_", "cover_" o timestamps "123456_".

Parameters

- `serverFileName` **string** Nombre del archivo tal como está guardado en el servidor

Returns **string** Nombre limpio y legible para el usuario

sendSuccess

Envía una respuesta JSON de éxito estandarizada. Formato: { message: string, data?: any } Útil para mantener consistencia en todas las respuestas exitosas de la API.

Parameters

- `res`
- `message` **string** Mensaje descriptivo del éxito
- `data` **any** Datos opcionales a devolver (objeto, array, etc.) (optional, default `null`)
- `statusCode` **number** Código HTTP de estado (200 OK por defecto) (optional, default `200`)

sendError

Envía una respuesta JSON de error estandarizada. Formato: { error: string } Útil para mantener consistencia en respuestas de error de la API.

Parameters

- `res`
- `errorMessage` **string** Mensaje descriptivo del error
- `statusCode` **number** Código HTTP de error (400 Bad Request por defecto) (optional, default `400`)

validateUserFields

Valida los campos de un usuario durante registro o actualización. Lanza un error con todos los mensajes concatenados si alguna validación falla.

Parameters

- `fields` **Object** Campos a validar
- `isUpdate` **boolean** true si es actualización (campos opcionales) (optional, default `false`)

- Throws **Error** Si hay errores de validación